

Документация по архитектуре набора инструкций F32a (ISA)

ISA F32a — это стековая архитектура набора инструкций, созданная для учебных целей. В этой документации приводится обзор инструкций, доступных в F32a ISA, их синтаксис и семантика.

Обзор архитектуры

F32a — это 32-битная стековая архитектура, вдохновлённая системами Forth. Она включает:

- два основных стека: стек данных и стек возвратов;
- два регистра общего назначения: A и B;
- операции, ориентированные на стек, которые упрощают многие задачи программирования;
- режим расширенной арифметики (EAM) для операций, зависящих от переноса;
- флаг переноса для арифметических операций;
- ввод-вывод с отображением в память (Memory-Mapped I/O).

Эта стековая архитектура предлагает другую парадигму программирования по сравнению с регистровыми архитектурами, поэтому отлично подходит для изучения альтернативных вычислительных моделей.

Комментарии в ассемблерном коде F32a обозначаются символом `\`.

Вдохновлено [F18a](#)

Специфичные представления состояния ISA

- `A:dec`, `A:hex` — регистр A.
- `B:dec`, `B:hex` — регистр B.
- `T:dec`, `T:hex` — вершина стека данных.
- `S:dec`, `S:hex` — второе значение в стеке данных.
- `R:dec`, `R:hex` — вершина стека возвратов.
- `stack:dec`, `stack:hex` — весь стек данных.
- `rstack:dec`, `rstack:hex` — весь стек возвратов.
- `EAM` — режим расширенной арифметики (1 — включён, 0 — выключен).
- `C` — флаг переноса (1 — установлен, 0 — сброшен).

Инструкции

Размер инструкции: 1 байт для опкода, 4 байта для каждого аргумента.

Режим расширенной арифметики (EAM)

Режим расширенной арифметики (EAM) позволяет выполнять сложение с битом переноса.

Подробности см. в инструкции `add`.

Флаг переноса

Большинство инструкций сбрасывает флаг переноса. Исключения: `add`, `dup`.

Инструкции перемещения данных

- **Literal**

- **Синтаксис:** `lit <value>`
- **Описание:** Поместить непосредственное значение в стек данных.
- **Операция:** `dataStack.push(<value>)`

- **Fetch from Address**

- **Синтаксис:** `@p <address>`
- **Описание:** Поместить значение из заданного адреса в стек данных.
- **Операция:** `dataStack.push(mem[<address>])`

- **Fetch**

- **Синтаксис:** `@`
- **Описание:** Поместить в стек данных значение из адреса, находящегося в регистре `A`.
- **Операция:** `dataStack.push(mem[A])`

- **Fetch Plus**

- **Синтаксис:** `@+`
- **Описание:** Поместить в стек данных значение из адреса в `A`, затем увеличить `A`.
- **Операция:** `dataStack.push(mem[A]); A <- A + 1`

- **Fetch B**

- **Синтаксис:** `@b`
- **Описание:** Поместить в стек данных значение из адреса, находящегося в регистре `B`.
- **Операция:** `dataStack.push(mem[B])`

- **Store to Address**

- **Синтаксис:** `!p <address>`
- **Описание:** Сохранить верхнее значение стека данных по заданному адресу.
- **Операция:** `mem[<address>] <- dataStack.pop()`

- **Store**

- **Синтаксис:** `!`
- **Описание:** Сохранить верхнее значение стека данных по адресу в регистре `A`.
- **Операция:** `mem[A] <- dataStack.pop()`

- **Store Plus**

- **Синтаксис:** `!+`
- **Описание:** Сохранить верхнее значение стека данных по адресу в `A`, затем увеличить `A`.
- **Операция:** `mem[A] <- dataStack.pop(); A <- A + 1`

- **Store B**

- **Синтаксис:** `!b`
- **Описание:** Сохранить верхнее значение стека данных по адресу в регистре `B`.
- **Операция:** `mem[B] <- dataStack.pop()`
- **A-Store**
 - **Синтаксис:** `a!`
 - **Описание:** Записать верхнее значение стека данных в регистр `A`.
 - **Операция:** `A <- dataStack.pop()`
- **B-Store**
 - **Синтаксис:** `b!`
 - **Описание:** Записать верхнее значение стека данных в регистр `B`.
 - **Операция:** `B <- dataStack.pop()`
- **A-Fetch**
 - **Синтаксис:** `a`
 - **Описание:** Поместить значение регистра `A` в стек данных.
 - **Операция:** `dataStack.push(A)`

Арифметические инструкции

- **Add**
 - **Синтаксис:** `+`
 - **Описание:** Сложить два верхних значения в стеке данных.
 - **Операция (без EAM):** `dataStack.push(dataStack.pop() + dataStack.pop())`,
установить флаг переноса
 - **Операция (с EAM):** `dataStack.push(dataStack.pop() + dataStack.pop() + carry)`,
установить флаг переноса
- **Multiply Step**
 - **Синтаксис:** `+`
 - **Описание:** Выполнить один шаг умножения.
 - **Операция:** `T <- T + (if A[0] then S else 0); A <- A >> 1; if T[0] then A[31] <- 1 else A[31] <- 0; T <- T >> 1`
- **Divide Step**
 - **Синтаксис:** `+/`
 - **Описание:** Выполнить один микро-шаг **восстанавливающего деления**.
 - Делимое — `A`
 - Делитель — `mem[B]`
 - Частичный остаток — `S`
 - Частное (частичный результат) — `T`
 - **Операция:**
 1. `S <- S << 1`
 2. `if A[31] then S[0] <- 1`

- 3. `A <- A << 1`
- 4. `if S >= mem[B] then S <- S - mem[B]`
- 5. `T <- T << 1`
- 6. `if S >= mem[B] then T[0] <- 1`

- **Пример на Python**

- **Left Shift**

- **Синтаксис:** `2*`
- **Описание:** Сдвинуть верхнее значение стека данных влево на 1 бит.
- **Операция:** `dataStack.push(dataStack.pop() << 1)`

- **Right Shift**

- **Синтаксис:** `2/`
- **Описание:** Сдвинуть верхнее значение стека данных вправо на 1 бит с сохранением знака (арифметический сдвиг).
- **Операция:** `dataStack.push(dataStack.pop() >> 1)`

- **Invert**

- **Синтаксис:** `inv`
- **Описание:** Инвертировать все биты верхнего значения стека данных.
- **Операция:** `dataStack.push(~dataStack.pop())`

- **Extend Arithmetic Instructions**

- **Синтаксис:** `eam`
- **Описание:** Извлечь значение из стека данных и установить флаг `EAM`.
- **Операция:** `eam <- dataStack.pop()`

Побитовые инструкции

- **And**

- **Синтаксис:** `and`
- **Описание:** Выполнить побитовое И над двумя верхними значениями стека данных.
- **Операция:** `dataStack.push(dataStack.pop() & dataStack.pop())`

- **Xor**

- **Синтаксис:** `xor`
- **Описание:** Выполнить побитовое исключающее ИЛИ над двумя верхними значениями стека данных.
- **Операция:** `dataStack.push(dataStack.pop() ^ dataStack.pop())`

Инструкции работы со стеком

- **Drop**

- **Синтаксис:** `drop`
- **Описание:** Удалить верхнее значение из стека данных.

- **Операция:** `dataStack.pop()`
- **Dup**
 - **Синтаксис:** `dup`
 - **Описание:** Дублировать верхнее значение стека данных.
 - **Операция:** `dataStack.push(dataStack.top())`, флаг переноса не изменяется
- **Over**
 - **Синтаксис:** `over`
 - **Описание:** Поменять местами два верхних значения стека данных.
 - **Операция:** `T <- dataStack.pop(); S <- dataStack.pop(); dataStack.push(T); dataStack.push(S)`

Инструкции управления потоком

- **Return**
 - **Синтаксис:** `;`
 - **Описание:** Возврат из подпрограммы.
 - **Операция:** `p <- returnStack.pop()`
- **Call**
 - **Синтаксис:** `<label>`
 - **Описание:** Вызвать подпрограмму по указанной метке.
 - **Операция:** `returnStack.push(p); p <- <label>`
- **Jump**
 - **Синтаксис:** `<label> ;`
 - **Описание:** Переход к указанной метке.
 - **Операция:** `p <- <label>`
- **Next**
 - **Синтаксис:** `next <label>`
 - **Описание:** Повторить цикл до указанной метки, уменьшая `R`.
 - **Операция:** `if R != 0 then R <- R - 1; p <- <label>`
- **If**
 - **Синтаксис:** `if <label>`
 - **Описание:** Перейти к указанной метке, если верхнее значение стека данных равно нулю.
 - **Операция:** `if dataStack.pop() == 0 then p <- <label>`
- **Minus If**
 - **Синтаксис:** `-if <label>`
 - **Описание:** Перейти к указанной метке, если верхнее значение стека данных неотрицательно.

- **Операция:** `if dataStack.pop() >= 0 then p <- <label>`

- **Halt**

- **Синтаксис:** `halt`
- **Описание:** Остановить машину.

Инструкции работы с регистрами

- **RIntoT**

- **Синтаксис:** `r>`
- **Описание:** Переместить верхнее значение стека возвратов в стек данных.
- **Операция:** `dataStack.push(returnStack.pop())`

- **TIntoR**

- **Синтаксис:** `>r`
- **Описание:** Переместить верхнее значение стека данных в стек возвратов.
- **Операция:** `returnStack.push(dataStack.pop())`